End-to-End Impact of Distributed Fine-Tuning Strategies on vLLM Inference for 1B and 3B LLMs

Comprehensive evaluation of three distributed fine-tuning strategies—PyTorch Distributed Data Parallel (DDP), DeepSpeed ZeRO Stage 2, and PyTorch Fully Sharded Data Parallel (FSDP)—and their impact the full lifecycle of large language model deployment, from multi-GPU training to inference with vLLM.

Group 25
Peter McMaster · pjm2188
Robert Solomon ·rs4759
Sudarshan Kadalazhi · sk5573
Damien Kim · ddk2145

GitHub: https://github.com/PeterMcMaster/distributed-llm-pipeline

Tracking:
DDP
DeepSpeed ZeRO-2
FSDP

Motivation & Problem

Why this matters

- Modern LLM fine-tuning is usually evaluated only on training speed and GPU memory, but deployment requires more than just a fast training run.
- In real systems, the training strategy affects the whole pipeline: checkpoint format, export/consolidation time, vLLM compatibility, model load time, latency, and inference throughput.
- Naively choosing the most memory-efficient method, like FSDP, may reduce GPU memory during training but introduce extra checkpoint/export complexity before the model can actually be served.
- For teams fine-tuning 1B–3B parameter LLMs, the question is not just “which trains fastest?” but which strategy gets a usable model into vLLM most efficiently.

Problem statement

We compare DDP, DeepSpeed ZeRO-2, and PyTorch FSDP for fine-tuning Llama 3.2 1B and 3B models on UltraChat200K, targeting the bottleneck between distributed training efficiency and vLLM deployment readiness.

Goal: Achieve the best training-to-serving pipeline by comparing tokens/sec, peak GPU memory, export time, vLLM load time, first-token latency, and throughput across DDP, ZeRO-2, and FSDP for Llama 3.2 1B/3B models.

Technical Approach — Model & Data

Model

Models: Meta Llama 3.2 1B and 3B
Task: Instruction fine-tuning
Framework: PyTorch + Hugging Face Transformers
Training strategies compared:

- PyTorch Distributed Data Parallel DDP
- DeepSpeed ZeRO-2
- PyTorch FSDP

Purpose of using 1B vs. 3B: test whether scaling model size changes the best training-to-vLLM deployment strategy.

Dataset

Dataset: UltraChat200K instruction-tuning dataset
Format: user-assistant conversational examples
Preprocessing: same tokenization, max sequence length, train/validation split, and batching across all methods
Evaluation target: fixed validation loss/checkpoint quality threshold, so systems metrics are compared fairly
Main focus: systems performance, not maximizing model quality.

Hardware

Training: Insomnia Cluster / runpod with 4 x A6000

vLLM serving: GCP Compute Engine with 1 x NVIDIA L4

Technical Approach — Optimizations Applied

1

DDP Baseline

Distributed Data Parallel

DDP gives each GPU a full model copy and synchronizes gradients with all-reduce.

PRO: Strong speed baseline

CON: Memory-inefficient for LLMs

2

DeepSpeed ZeRO-2

Optimizer/Gradient Sharding

Shards states across GPUs, reducing redundancy while keeping export paths manageable.

PRO: Better memory efficiency than DDP

CON: Larger operational overhead (checkpointing, NCCL tuning and multi-GPU failures harder to debug)

3

PyTorch FSDP

Fully Sharded Data Parallel

Shards parameters, gradients, and optimizer states across all GPUs for maximum memory efficiency.

PRO: Highest memory savings; enables training larger models on the same hardware; reduces redundancy across all model states

CON: Requires careful wrapping and configuration; complex export orchestration

Distributed Data Parallel (DDP) Implementation

DDP Overview

Replicates model across GPUs, splitting batches and synchronizing gradients via NCCL all-reduce.

- One process per GPU
- Faster training & larger batches
- Better GPU utilization
- Scalable fine-tuning (1B & 3B)

What We Implemented

Distributed fine-tuning using HF Transformers, TRL SFTTrainer, and Slurm.

- 2-GPU & 4-GPU (A6000) setups
- Mixed-precision BF16 training
- Sharded training data across nodes
- HF-compatible model exports

Systems Challenges

- **Slurm Reliability:** Frequent node failures; solved by manual targeting of stable nodes.
- **Storage Quotas:** DDP checkpointing caused "Disk quota exceeded" errors.
- **Serialization:** Model/optimizer state overhead for 3B parameter runs.

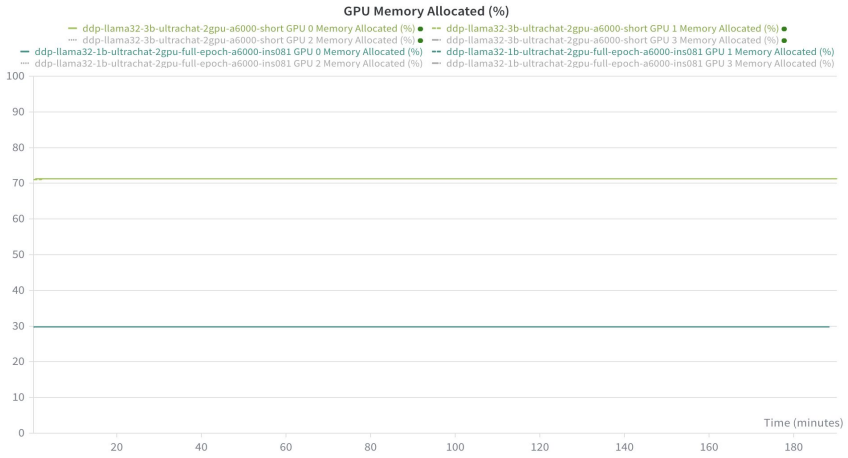
Iterative Training Strategy

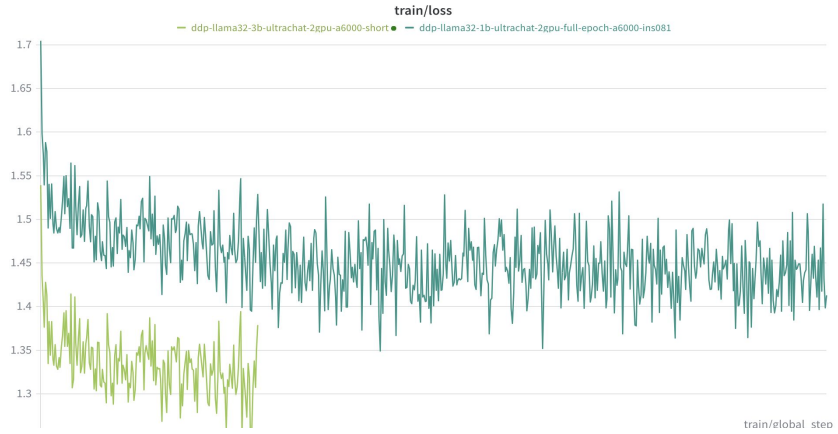
- **Phase 1 (Short Runs):** Verified sync, reduced steps, and confirmed export correctness.
- **Phase 2 (Full-Epoch):** Removed limits once scaling behavior was validated.
- **Benefit:** Faster debugging & reduced storage pressure.

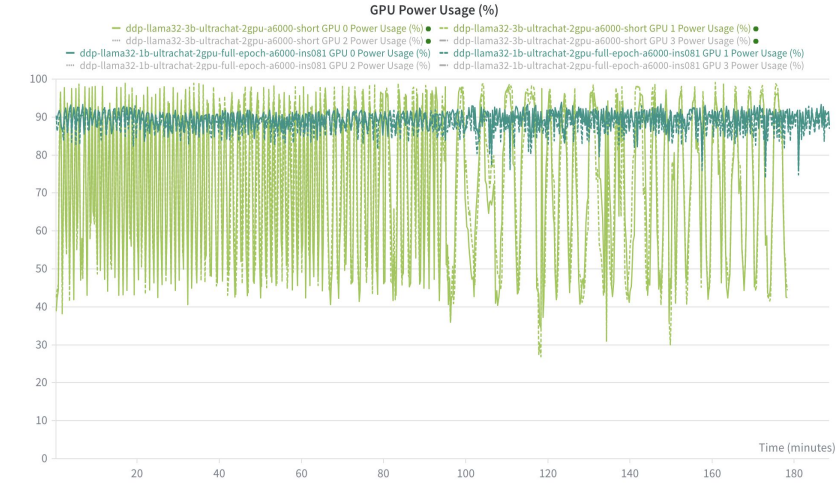
Stability & Optimizations

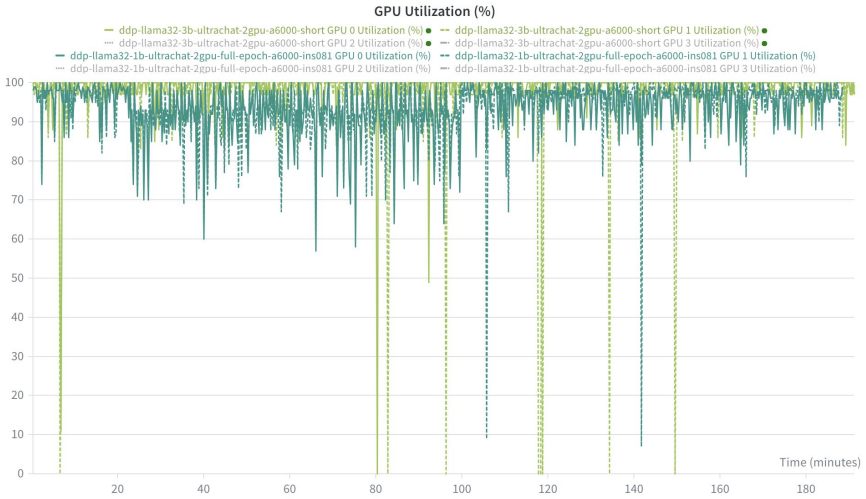
- **Disabled:** Intermediate checkpoints and evaluation during training.
- **Enabled:** BF16 mixed precision for reduced memory footprint.
- **Tuned:** NCCL communication settings for runtime stability.
- **Result:** Successfully produced scalable, reproducible infrastructure.

Distributed Data Parallel (DDP) Implementation









- The power usage for the 3B model fluctuated a lot more
- The training loss for 3B model was a lot lower, but both stablized within 2k-3k steps
- DDP uses the GPU very well, with GPU utilization for both models being very close to 100%

DeepSpeed ZeRO-2 Implementation

ZeRO-2 Overview

Replicates model across GPUs, but shards optimizer states and gradients (to reduce memory overhead)

- Lower memory usage than plain DDP
- Supports larger models and batch sizes on the same hardware
- Easier to export, since the final model weights are not fully sharded across GPUs

What We Implemented

Distributed fine-tuning using HF Transformers, DeepSpeed ZeRO-2, and Slurm.

- 4-GPU (A6000) setup
- Mixed-precision BF16 training
- NCCL-based multi-GPU communication
- Step-based checkpointing (for recovery and export)

Systems Challenges

- **Node Reliability:** some multi-GPU jobs hung during distributed startup/NCCL initialization, so we identified stable nodes to target.
- **Storage Quotas:** frequent DeepSpeed checkpoints became too very (Especially for the 3B runs)
- **Environment Setup:** jobs required handling of tokens and credentials (WandB, HF), and reproducible package versions

Iterative Training Strategy

- **Phase 1 (Smoke Test):** Verified CUDA visibility, DeepSpeed launch, distributed sync, and short training run.
- **Phase 2 (Stability Tuning):** Debugged hanging behaviour, tuned NCCL settings and found stable, reliable nodes
- **Phase 3 (Training):** Confirmed stable ZeRO-2 training and export on 4X A6000, as well as adjusted the checkpoint strategy for the 3B run
- **Benefit:** Reduced debugging time and wasted runs by validating systems behaviour, before committing to larger training runs

Stability & Optimizations

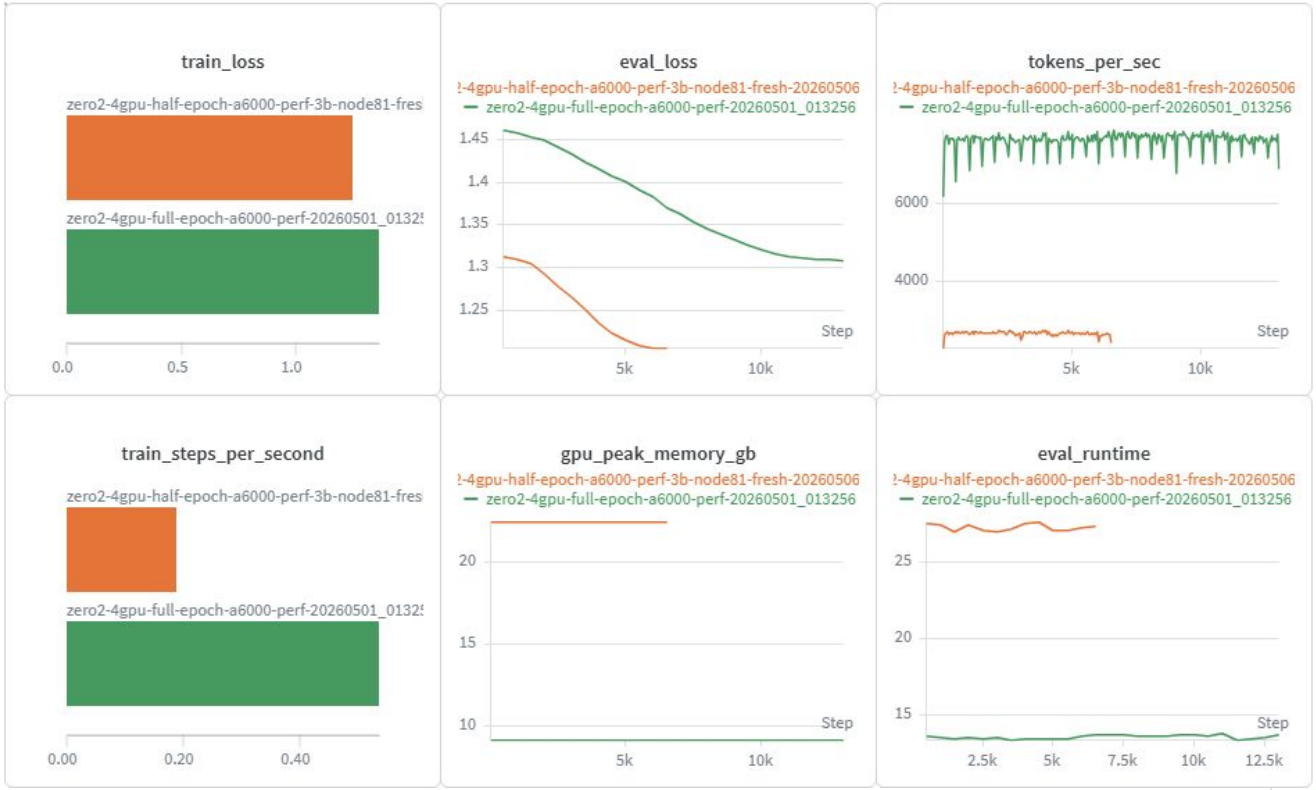
- **Enabled:** BF16 mixed precision to reduce memory usage
- **Adjusted:** Checkpoint frequency / retention to reduce storage overhead
- **Used:** Manual node targeting for stable A6000 hosts
- **Limited:** 3B experiments to half-epoch runs, to control runtime and systems risk
- **Result:** Stable ZeRO-2 training at both 1B and 3B scales

ZeRO2 finetuning

Robert Solomon

Last updated: May 9, 2026

DeepSpeed ZeRO-2 Report



- 3B has lower evaluation loss (even with only half an epoch)
- Higher tokens per second and training steps per second for the smaller 1B model
- Higher compute cost for the 3B model (almost 3x peak memory usage difference)

PyTorch FSDP Implementation

FSDP Overview

Shards parameters, gradients, and optimizer states across GPUs to maximize memory efficiency and enable larger-model fine-tuning.

- One process per GPU
- Supports larger models on the same hardware (1B & 3B)
- Reduces redundant memory across ranks
- Allows higher effective batch sizes under the same VRAM limits

What We Implemented

Distributed fine-tuning using HF Transformers + PyTorch FSDP.

- 4-GPU (A6000) setup
- Mixed-precision BF16 training
- Auto-wrap policies for transformer layers
- FSDP-compatible checkpointing and HF export path
- Sharded optimizer + parameter states

Systems Challenges

- FSDP requires explicit wrapping policies for stable sharding
- Exporting consolidated weights requires gathering sharded states
- **Larger configuration surface** (activation checkpointing, prefetching, all-gather limits)
- **Higher sensitivity** to NCCL communication settings

Iterative Training Strategy

- **Phase 1 (Minimal Local Runs):** Validated core FSDP behavior on a single GPU
- **Phase 2 (Diagnosis & Infrastructure Alignment):** Focused on making FSDP actually work on constrained multi-GPU hardware
- **Phase 3 (Full Training):** Once stability and scaling behavior were confirmed, transitioned to full-length fine-tuning
- **Benefit:** Reduced iteration time while ensuring FSDP correctness and stability before committing to long, resource-intensive runs

Stability & Optimizations

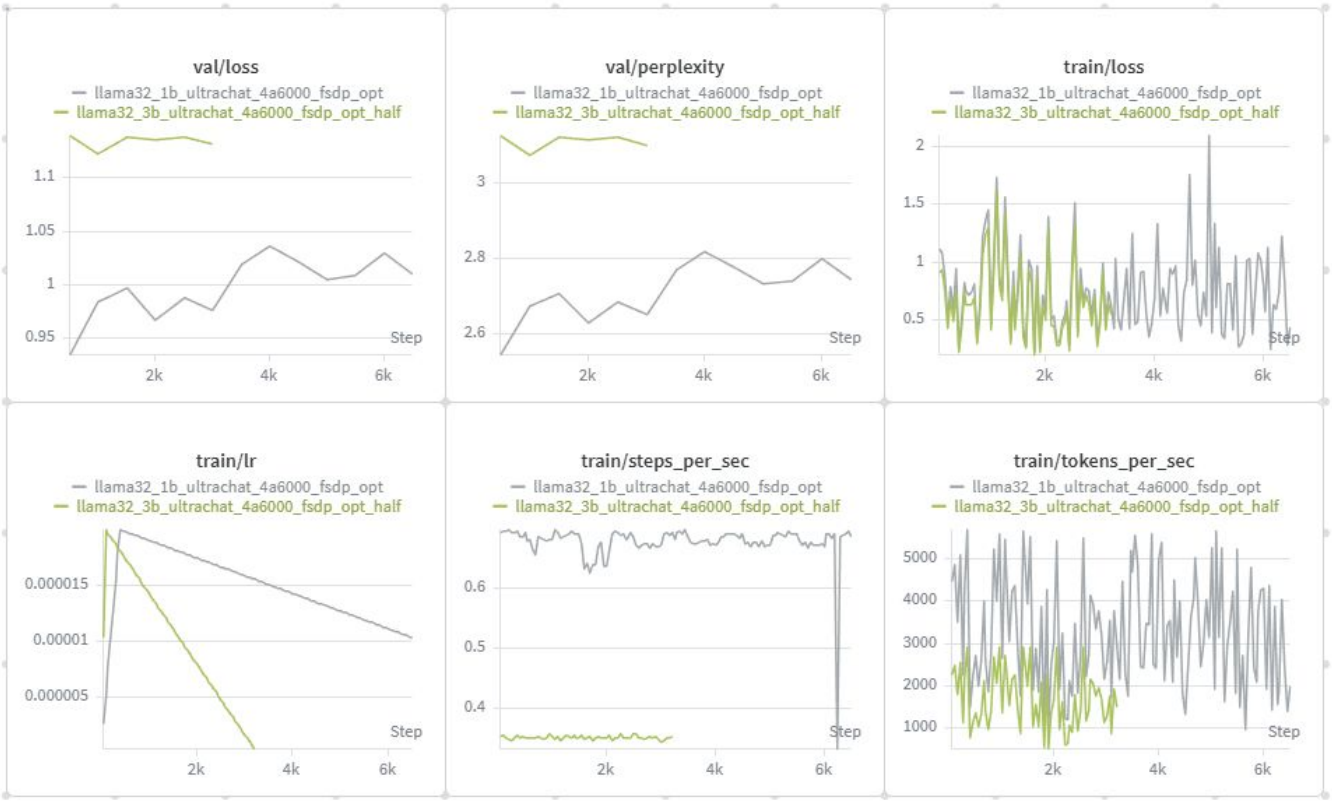
- **Enabled:** BF16 mixed precision for reduced memory footprint
- **Enabled:** FSDP prefetching + limit_all_gathers for smoother communication
- **Tuned:** NCCL settings for multi-GPU sharded collectives
- **Result:** Stable, scalable fine-tuning for 1B and 3B models with minimal VRAM overhead

PyTorch FSDP Implementation

FSDP Training Curves (1B vs 3B)

Training curves for 1B and 3B FSDP runs showing stable loss reduction, consistent throughput, and correct LR scheduling across 6K+ steps.

Damien Kim



- Stable loss/perplexity curves for both 1B and 3B
- Consistent throughput (steps/sec, tokens/sec) across training
- Correct LR schedule + no divergence under BF16 FSDP

Profiling Methodology

Tools used, what you measured, how you ensured measurements are stable.

Tools

- vLLM logs
- nvidia-smi GPU traces
- iostat
- htop
- Per run JSON/CSV summaries
- Weights & Biases dashboard per training strategy

Metrics captured

- Server load time
- Average end-to-end latency
- Average first-token latency
- Completion tokens per second
- Peak GPU utilization
- Peak GPU memory used
- Peak power draw
- Checkpoint export / conversion overhead

Results — Headline Numbers

3x

increased export time / complexity for FSDP

compared to DDP, DeepSpeed ZeRO-2

2.5x

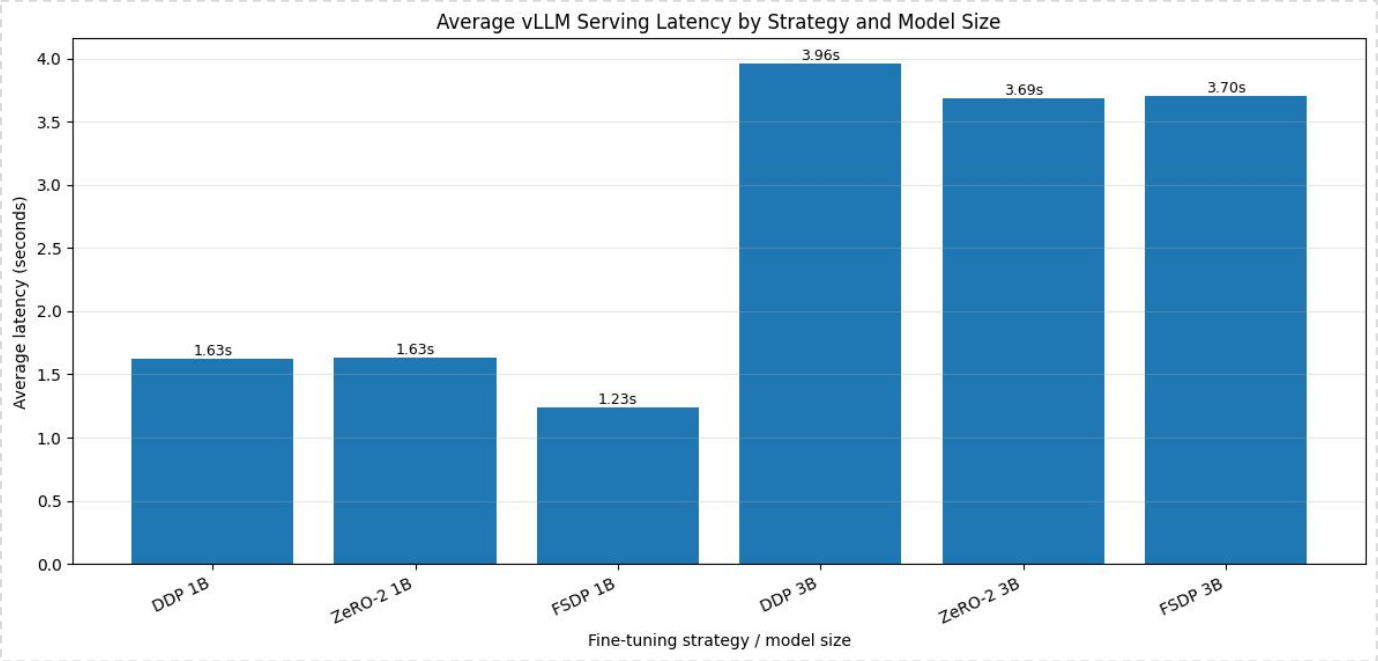
lower latency with 1B models

2.4x

higher throughout with 1B models

Results and Serving Evidence

Insert profiler trace screenshot, GPU timeline, or operator-level breakdown image.



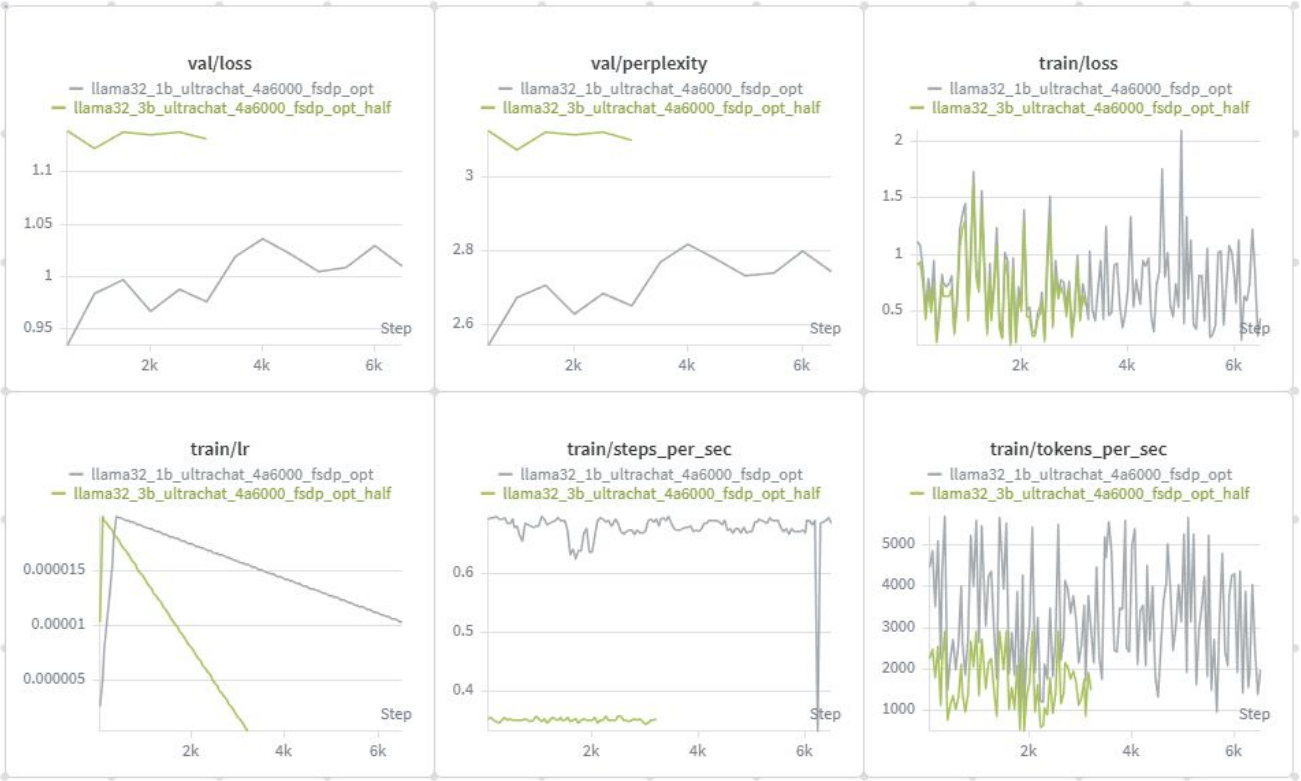
vLLM Serving Evidence: Full Experiment Summary

Run	Load (s)	Avg latency (s)	First token (s)	Tok/s	Peak mem (GB)	Peak util (%)
DDP 1B	62.02	1.63	0.02	80.63	19.03	100.00
ZeRO-2 1B	82.03	1.63	0.02	80.55	19.03	100.00
FSDP 1B	82.03	1.23	0.02	78.16	19.03	100.00
DDP 3B	146.09	3.96	0.05	32.56	18.67	100.00
ZeRO-2 3B	132.06	3.69	0.04	34.44	18.67	100.00
FSDP 3B	124.04	3.70	0.04	34.32	18.67	100.00

- 3B models have higher latency because each generated token requires more transformer computation than 1B models.
- Training strategy affects the final checkpoint, but model size is the clearest driver of inference latency.
- Once checkpoints are exported to a standard Hugging Face format and served through vLLM, the original distributed training strategy has little direct effect on serving performance. Model size is the dominant factor.
- Serving latency is driven more by model size, GPU hardware, sequence length, batch size, and vLLM configuration than by the original training strategy.

Demo / Visualization

```
(petermcmaster) petermcmaster@instance-20260405-212143:~$ nvidia-smi
Mon Apr  6 15:20:34 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.126.20             Driver Version: 580.126.20     CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L4                      Off |   00000000:00:03.0 Off |                    0 |
| N/A   36C    P8             13W /   72W |       0MiB /  23034MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
(petermcmaster) petermcmaster@instance-20260405-212143:~$ vllm --help
usage: vllm [-h] [-v] {chat,complete,serve,launch,bench,collect-env,run-batch} ...

vLLM CLI

positional arguments:
  {chat,complete,serve,launch,bench,collect-env,run-batch}
    chat                Generate chat completions via the running API server.
    complete            Generate text completions based on the given prompt via the running API server.
    serve               Launch a local OpenAI-compatible API server to serve LLM completions via HTTP.
    launch              Launch individual vLLM components.
    bench               vLLM bench subcommand.
    collect-env         Start collecting environment information.
    run-batch           Run batch prompts and write results to file.

options:
  -h, --help            show this help message and exit
  -v, --version         show program's version number and exit

For full list:            vllm [subcommand] --help=all
For a section:            vllm [subcommand] --help=ModelConfig    (case-insensitive)
For a flag:               vllm [subcommand] --help=max-model-len  (_ or - accepted)
Documentation:            https://docs.vllm.ai
```




```
petermcmaster@instance-20260405-212143:~$ curl http://localhost:8000/v1/chat/completions \
>   -H "Content-Type: application/json" \
>   -H "Authorization: Bearer devkey" \
>   -d '{
>     "model": "meta-llama/Llama-3.2-1B-Instruct",
>     "messages": [
>       {"role": "user", "content": "Say hello in one sentence."}
>     ],
>     "temperature": 0.0,
>     "max_tokens": 32
>   }'
{"id":"chatcmpl-89637f3c01a0e57b","object":"chat.completion","created":1775489808,"model":"meta-llama/Llama-3.2-1B-Instruct","choices":[{"index":0,"message":{"role":"assistant","content":"Hello, how can I
assist you today?","refusal":null,"annotations":null,"audio":null,"function_call":null,"tool_calls":[],"reasoning":null},"logprobs":null,"finish_reason":"stop","stop_reason":null,"token_ids":null}],"servic
e_tier":null,"system_fingerprint":null,"usage":{"prompt_tokens":41,"total_tokens":51,"completion_tokens":10,"prompt_tokens_details":null},"prompt_logprobs":null,"prompt_token_ids":null,"kv_transfer_params"
:null}petermcmaster@instance-20260405-212143:~$
```

Lessons Learned

Time budget: 1 minute. What surprised you? What didn't work? Be honest.

What worked

- vLLM deployment and serving was straightforward for HF models
- FSDP
  - FSDP enabled stable multi-GPU training
  - Successfully exported HuggingFace-compatible checkpoints
- DeepSpeed ZeRO-2
  - Larger model improved eval loss but with a major throughput and memory cost
  - Systems engineering mattered just as much as model code

What didn't work

- 405B on 4 x A6000 was not feasible
- FSDP
  - Process management complexity made distributed training and export difficult to monitor and control
- DeepSpeed ZeRO-2
  - Frequent checkpointing created major storage pressure (especially for 3B)
  - Some runs would hang during distributed startup (mostly an issue with the node)

Next Steps

- **Extend to multi-node serving**
  - Scale vLLM tensor parallelism to 2+ nodes for 7B+ models
  - Measure cross-node communication overhead vs single-node batching gains
  - Identify the model-size threshold where multi-node becomes cost-effective
- **Evaluate larger models**
  - Repeat the DDP/FSDP/ZeRO-2 pipeline on Llama-7B and 13B to quantify :
    - where DDP runs OOM
    - whether FSDP/ZeRO-2 training speed justifies checkpoint export complexity
    - measure total end-to-end time including conversion
- **Compare full fine-tuning vs. LoRA/QLoRA**
  - Measure training time, memory footprint, checkpoint export overhead, and final vLLM inference quality (MMLU/HellaSwag) to validate whether adapter-based methods achieve acceptable quality with better deployment efficiency

Teamwork & Contributions

Time budget: 30 seconds. Who did what. Required slide — graded individually.

Member	Primary contributions	Tools / artifacts
Peter McMaster	vLLm setup, deployment, and checkpoint evaluation	/vllm_deployment_eval
Robert Solomon	DeepSpeed ZeRO-2 implementation & scaling	/ZeRO2_training (training scripts, Slurm wrappers, configs) W&B report HF Models (1B & 3B)
Sudarshan Kadalazhi	[e.g., Quantization, evaluation harness]	/DDP directory W&B report HF Models (1B & 3B)
Damien Kim	PyTorch FSDP implementation & scaling	/FSDP directory (training scripts, configs, logs, exports) HF Models (1B & 3B) wandb report (1B & 3B)

Collaboration: shared group chat, code review on every PR.

Thank you

Questions?

GitHub: https://github.com/PeterMcMaster/distributed-llm-pipeline

Tracking:
DDP
DeepSpeed ZeRO-2
FSDP